XERNIX

# LeetCode Revision Notes

Simple layout • complete statements • readable code

## 50 problems

Made for revision: read the whole problem, inspect every example, check the constraints and complexity, then reproduce the solution.

**Clean typography • comfortable spacing • large code**

# 0001. 1. Two Sum

Easy • C++

## QUESTION

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

## EXAMPLES

### Example 1

```
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
```

### Example 2

```
Input: nums = [3,2,4], target = 6
Output: [1,2]
```

### Example 3

```
Input: nums = [3,3], target = 6
Output: [0,1]
```

## CONSTRAINTS

- 2 <= nums.length <= 104
- -109 <= nums[i] <= 109
- -109 <= target <= 109
- Only one valid answer exists.

## COMPLEXITY

Time: O(n) average Space: O(n)

## SOLUTION

```cpp
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> mp;
        for (int i = 0; i < nums.size(); i++) {
            int x = target - nums[i];
            if (mp.find(x) != mp.end()) {
                return {mp[x], i};
            }
            mp[nums[i]] = i;
        }
        return {};
    }
};
```

# 0002. 2. Add Two Numbers

Medium • C++

## QUESTION

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

## EXAMPLES

### Example 1

```
Input: l1 = [2,4,3], l2 = [5,6,4]
Output: [7,0,8]
Explanation: 342 + 465 = 807.
```

### Example 2

```
Input: l1 = [0], l2 = [0]
Output: [0]
```

### Example 3

```
Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]
Output: [8,9,9,9,0,0,0,1]
```

## CONSTRAINTS

- The number of nodes in each linked list is in the range [1, 100].
- 0 <= Node.val <= 9
- It is guaranteed that the list represents a number that does not have leading zeros.

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        ListNode* dummy = new ListNode(0);
        ListNode* temp = dummy;
        int carry = 0;
        while (l1 != NULL || l2 != NULL || carry != 0) {
            int sum = carry;
            if (l1 != NULL) {
                sum += l1->val;
                l1 = l1->next;
            }
            if (l2 != NULL) {
                sum += l2->val;
                l2 = l2->next;
            }
            carry = sum / 10;
            temp->next = new ListNode(sum % 10);
            temp = temp->next;
        }
        return dummy->next;
    }
};
```

# 0003. 3. Longest Substring Without Repeating Characters

Medium • C++

## QUESTION

Given a string s, find the length of the longest substring without duplicate characters.

## EXAMPLES

### Example 1

```
Input: s = "abcabcbb"
Output: 3
Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correc
```

### Example 2

```
Input: s = "bbbbb"
Output: 1
Explanation: The answer is "b", with the length of 1.
```

### Example 3

```
Input: s = "pwwkew"
Output: 3
Explanation: The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.
```

## CONSTRAINTS

• 0 <= s.length <= 5 * 104

• s consists of English letters, digits, symbols and spaces.

## COMPLEXITY

Time: O(n) average Space: O(n)

## SOLUTION

```cpp
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        unordered_set<char> st;
        int left = 0;
        int maxi = 0;
        for (int right = 0;right<s.size();right++){
            while(st.find(s[right])!=st.end()){
                st.erase(s[left]);
                left++;
            }
            st.insert(s[right]);
            maxi = max(maxi,right-left+1);
        }
        return maxi;
    }
};
```

# 0004. 4. Median of Two Sorted Arrays

Hard • C++

## QUESTION

Given two sorted arrays nums1 and nums2 of size m and n respectively, return the median of the two sorted arrays.

The overall run time complexity should be O(log (m+n)).

## EXAMPLES

### Example 1

```
Input: nums1 = [1,3], nums2 = [2]
Output: 2.00000
Explanation: merged array = [1,2,3] and median is 2.
```

### Example 2

```
Input: nums1 = [1,2], nums2 = [3,4]
Output: 2.50000
Explanation: merged array = [1,2,3,4] and median is (2 + 3) / 2 = 2.5.
```

## CONSTRAINTS

- nums1.length == m
- nums2.length == n
- 0 <= m <= 1000
- 0 <= n <= 1000
- 1 <= m + n <= 2000
- -106 <= nums1[i], nums2[i] <= 106

## COMPLEXITY

Time: O(n log n) Space: O(n)

## SOLUTION

```cpp
class Solution {
public:
    double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
        nums1.insert(nums1.end(),nums2.begin(),nums2.end());
        sort(nums1.begin(), nums1.end());
        if(nums1.size()%2==0){
            return (nums1[(nums1.size()/2)-1]+nums1[nums1.size()/2])/2.0;
        }
        else{
            return nums1[nums1.size()/2.0];
        }
        return{};
    }
};
```

# 0007. 7. Reverse Integer

Medium • C++

## QUESTION

Given a signed 32-bit integer x, return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range [-231, 231 - 1], then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

## EXAMPLES

### Example 1

```
Input: x = 123
Output: 321
```

### Example 2

```
Input: x = -123
Output: -321
```

### Example 3

```
Input: x = 120
Output: 21
```

## CONSTRAINTS

• -231 <= x <= 231 - 1

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
 1  class Solution {
 2  public:
 3      int reverse(int x) {
 4          int rev = 0;
 5
 6          while(x!=0){
 7              int d = x%10;
 8              if(rev > INT_MAX/10 || rev < INT_MIN/10)
 9                  return 0;
10              rev = rev*10 + d;
11              x=x/10;
12          }
13          return rev;
14      }
15  };
```

# 0009. 9. Palindrome Number

Easy • C++

## QUESTION

Given an integer x, return true if x is a palindrome, and false otherwise.

## EXAMPLES

### Example 1

```
Input: x = 121
Output: true
Explanation: 121 reads as 121 from left to right and from right to left.
```

### Example 2

```
Input: x = -121
Output: false
Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it
```

### Example 3

```
Input: x = 10
Output: false
Explanation: Reads 01 from right to left. Therefore it is not a palindrome.
```

## CONSTRAINTS

• -231 <= x <= 231 - 1

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
class Solution {
public:
    bool isPalindrome(int x){
        if (x < 0)
            return false;
        if (x != 0 && x % 10 == 0)
            return false;
        int rev = 0;
        while (x > rev) {
            rev = rev * 10 + x % 10;
            x /= 10;
        }
        if(x==rev || x==rev/10)
            return true;
        else
            return false;
    }
};
```

# 0011. 11. Container With Most Water

Medium • C++

## QUESTION

You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]).

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

## EXAMPLES

### Example 1

```
Input: height = [1,8,6,2,5,4,8,3,7]
Output: 49
Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case,
```

### Example 2

```
Input: height = [1,1]
Output: 1
```

## CONSTRAINTS

• n == height.length

• 2 <= n <= 105

• 0 <= height[i] <= 104

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    int maxArea(vector<int>& height) {
        int left =0;
        int right = height.size()-1;
        int area = 0;
        while(left<right){
            int h = min(height[left],height[right]);
            int a = h*(right-left);
            area = max(a,area);
            if(height[left]<height[right]){
                left++;
            }
            else{
                right--;
            }
        }
        return area;
    }
};
```

# 0014. 14. Longest Common Prefix

Easy • C++

## QUESTION

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string "".

## EXAMPLES

### Example 1

```
Input: strs = ["flower","flow","flight"]
Output: "fl"
```

### Example 2

```
Input: strs = ["dog","racecar","car"]
Output: ""
Explanation: There is no common prefix among the input strings.
```

## CONSTRAINTS

- 1 <= strs.length <= 200
- 0 <= strs[i].length <= 200
- strs[i] consists of only lowercase English letters if it is non-empty.

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    string longestCommonPrefix(vector<string>& strs) {

        string prefix = strs[0];

        for(int i = 1; i < strs.size(); i++) {

            int j = 0;

            while(j < prefix.size() &&
                  j < strs[i].size() &&
                  prefix[j] == strs[i][j]) {
                j++;
            }

            prefix = prefix.substr(0, j);

            if(prefix == "")
                return "";
        }

        return prefix;
    }
};
```

# 0019. 19. Remove Nth Node From End of List

Medium • C++

## QUESTION

Given the head of a linked list, remove the nth node from the end of the list and return its head.

Follow up: Could you do this in one pass?

## EXAMPLES

### Example 1

```
Input: head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]
```

### Example 2

```
Input: head = [1], n = 1
Output: []
```

### Example 3

```
Input: head = [1,2], n = 1
Output: [1]
```

## CONSTRAINTS

• The number of nodes in the list is sz.

• 1 <= sz <= 30

• 0 <= Node.val <= 100

• 1 <= n <= sz

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode *dummy = new ListNode(0);
        dummy->next = head;
        ListNode *temp=head;
        int count = 0;
        while(temp!=NULL){
            temp = temp->next;
            count++;
        }
        int k=count-n;
        temp = dummy;
        while(k>0){
            temp=temp->next;
            k--;
        }
        ListNode *temp1=temp->next;
        temp->next=temp1->next;
        delete temp1;
        ListNode* newHead = dummy->next;
        delete dummy;
        return newHead;
    }
};
```

# 0021. 21. Merge Two Sorted Lists

Easy • C++

## QUESTION

You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list.

## EXAMPLES

### Example 1

```
Input: list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]
```

### Example 2

```
Input: list1 = [], list2 = []
Output: []
```

### Example 3

```
Input: list1 = [], list2 = [0]
Output: [0]
```

## CONSTRAINTS

• The number of nodes in both lists is in the range [0, 50].

• -100 <= Node.val <= 100

• Both list1 and list2 are sorted in non-decreasing order.

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        if(list1==NULL){return list2;}
        if(list2==NULL){return list1;}
        ListNode* dummy=new ListNode(-1);
        ListNode* temp = dummy;
        while(list1!=NULL && list2!=NULL){
            if(list1->val<=list2->val){
                temp->next=list1;
                list1=list1->next;
            }
            else{
                temp->next=list2;
                list2=list2->next;
            }
            temp=temp->next;
        }
        if(list1!=NULL){
            temp->next=list1;
        }
        else{
            temp->next=list2;
        }
        return dummy->next;
    }
};
```

# 0024. 24. Swap Nodes in Pairs

Medium • C++

## QUESTION

Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Input: head = [1,2,3,4]

Output: [2,1,4,3]

Explanation:

Input: head = []

Output: []

Input: head = [1]

Output: [1]

Input: head = [1,2,3]

Output: [2,1,3]

## CONSTRAINTS

• The number of nodes in the list is in the range [0, 100].

• 0 <= Node.val <= 100

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        ListNode* dummy = new ListNode(0);
        dummy->next = head;
        ListNode* temp = dummy;
        while(temp->next != NULL && temp->next->next != NULL) {
            ListNode* first = temp->next;
            ListNode* second = first->next;
            first->next = second->next;
            second->next = first;
            temp->next = second;
            temp = first;
        }
        ListNode* newHead = dummy->next;
        delete dummy;
        return newHead;
    }
};
```

# 0025. 25. Reverse Nodes in k-Group

Hard • C++

## QUESTION

Given the head of a linked list, reverse the nodes of the list k at a time, and return the modified list.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.

You may not alter the values in the list's nodes, only nodes themselves may be changed.

## EXAMPLES

### Example 1

```
Input: head = [1,2,3,4,5], k = 2
Output: [2,1,4,3,5]
```

### Example 2

```
Input: head = [1,2,3,4,5], k = 3
Output: [3,2,1,4,5]
```

## CONSTRAINTS

• The number of nodes in the list is n.

• 1 <= k <= n <= 5000

• 0 <= Node.val <= 1000

## FOLLOW-UP

Can you solve the problem in O(1) extra memory space?

## COMPLEXITY

Time: $O(n^2)$ Space: O(1)

## SOLUTION

```cpp
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* reverseKGroup(ListNode* head, int k) {
        ListNode* temp = head;
        // Check whether k nodes exist
        for (int i = 0; i < k; i++) {
            if (temp == NULL)
                return head;
            temp = temp->next;
        }
        // Reverse first k nodes
        ListNode* prev = NULL;
        ListNode* curr = head;
        for (int i = 0; i < k; i++) {
            ListNode* next = curr->next;
            curr->next = prev;
            prev = curr;
            curr = next;
        }
        // head is now the last node of this group
        head->next = reverseKGroup(curr, k);
        return prev;
    }
};
```

# 0033. 33. Search in Rotated Sorted Array

Medium • C++

## QUESTION

There is an integer array nums sorted in ascending order (with distinct values).

Prior to being passed to your function, nums is possibly left rotated at an unknown index k (1 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed). For example, [0,1,2,4,5,6,7] might be left rotated by 3 indices and become [4,5,6,7,0,1,2].

Given the array nums after the possible rotation and an integer target, return the index of target if it is in nums, or -1 if it is not in nums.

You must write an algorithm with O(log n) runtime complexity.

## EXAMPLES

### Example 1

```
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4
```

### Example 2

```
Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1
```

### Example 3

```
Input: nums = [1], target = 0
Output: -1
```

## CONSTRAINTS

- 1 <= nums.length <= 5000
- -104 <= nums[i] <= 104
- All values of nums are unique.
- nums is an ascending array that is possibly rotated.
- -104 <= target <= 104

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    int search(vector<int>& nums, int target) {
        for(int i=0;i<nums.size();i++){
            if(nums[i]==target){
                return i;
            }
        }
        return -1;
    }
};
```

# 0034. 34. Find First and Last Position of Element in Sorted Array

Medium • C++

## QUESTION

Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value.

If target is not found in the array, return [-1, -1].

You must write an algorithm with O(log n) runtime complexity.

## EXAMPLES

### Example 1

```
Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
```

### Example 2

```
Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]
```

### Example 3

```
Input: nums = [], target = 0
Output: [-1,-1]
```

## CONSTRAINTS

• 0 <= nums.length <= 105

• -109 <= nums[i] <= 109

• nums is a non-decreasing array.

• -109 <= target <= 109

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        int first = -1;
        int last = -1;
        // Find first occurrence
        int left = 0;
        int right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                first = mid;
                right = mid - 1;       // search left
            }
            else if (target > nums[mid]) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }
        // Find last occurrence
        left = 0;
        right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                last = mid;
                left = mid + 1;        // search right
            }
            else if (target > nums[mid]) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }
        return {first, last};
    }
};
```

# 0035. 35. Search Insert Position

Easy • C++

## QUESTION

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with O(log n) runtime complexity.

## EXAMPLES

### Example 1

```
Input: nums = [1,3,5,6], target = 5
Output: 2
```

### Example 2

```
Input: nums = [1,3,5,6], target = 2
Output: 1
```

### Example 3

```
Input: nums = [1,3,5,6], target = 7
Output: 4
```

## CONSTRAINTS

- 1 <= nums.length <= 104
- -104 <= nums[i] <= 104
- nums contains distinct values sorted in ascending order.
- -104 <= target <= 104

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size()-1;
        while(left<=right){
            int mid = left + (right - left)/2;
            if(nums[mid]==target){
                return mid;
            }
            if(target>nums[mid]){
                left=mid+1;
            }
            else{
                right=mid-1;
            }
        }
        return left;
    }
};
```

# 0041. 41. First Missing Positive

Hard • C++

## QUESTION

Given an unsorted integer array nums. Return the smallest positive integer that is not present in nums.

You must implement an algorithm that runs in O(n) time and uses O(1) auxiliary space.

## EXAMPLES

### Example 1

```
Input: nums = [1,2,0]
Output: 3
Explanation: The numbers in the range [1,2] are all in the array.
```

### Example 2

```
Input: nums = [3,4,-1,1]
Output: 2
Explanation: 1 is in the array but 2 is missing.
```

### Example 3

```
Input: nums = [7,8,9,11,12]
Output: 1
Explanation: The smallest positive integer 1 is missing.
```

## CONSTRAINTS

• 1 <= nums.length <= 105

• -231 <= nums[i] <= 231 - 1

## COMPLEXITY

Time: O(n) average Space: O(n)

## SOLUTION

```cpp
class Solution {
public:
    int firstMissingPositive(vector<int>& nums) {
        unordered_set<int> st;
        for(int x:nums){
            st.insert(x);
        }
        for(int i =1;i<=nums.size();i++){
            if(st.find(i)==st.end()){
                return i;
            }
        }
        return nums.size()+1;
    }
};
```

# 0049. 49. Group Anagrams

Medium • C++

## QUESTION

Given an array of strings strs, group the anagrams together. You can return the answer in any order.

Input: strs = ["eat","tea","tan","ate","nat","bat"]

Output: [["bat"],["nat","tan"],["ate","eat","tea"]]

Explanation:

Input: strs = [""]

Output: [[""]]

Input: strs = ["a"]

Output: [["a"]]

## CONSTRAINTS

- 1 <= strs.length <= 104
- 0 <= strs[i].length <= 100
- strs[i] consists of lowercase English letters.

## COMPLEXITY

Time: O(n log n) Space: O(n)

## SOLUTION

```
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string,vector<string>>mp;
        for(string s:strs){
            string key = s;
            sort(key.begin(),key.end());
            mp[key].push_back(s);
        }
        vector<vector<string>> ans;
        for(auto x:mp){
            ans.push_back(x.second);
        }
        return ans;
    }
};
```

# 0053. 53. Maximum Subarray

Medium • C++

## QUESTION

Given an integer array nums, find the subarray with the largest sum, and return its sum.

Follow up: If you have figured out the O(n) solution, try coding another solution using the divide and conquer approach, which is more subtle.

## EXAMPLES

### Example 1

```
Input: nums = [-2,1,-3,4,-1,2,1,-5,4]
Output: 6
Explanation: The subarray [4,-1,2,1] has the largest sum 6.
```

### Example 2

```
Input: nums = [1]
Output: 1
Explanation: The subarray [1] has the largest sum 1.
```

### Example 3

```
Input: nums = [5,4,-1,7,8]
Output: 23
Explanation: The subarray [5,4,-1,7,8] has the largest sum 23.
```

## CONSTRAINTS

• 1 <= nums.length <= 105

• -104 <= nums[i] <= 104

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int sum = 0;
        int maxsum = INT_MIN;
        for(int i = 0;i<nums.size();i++){
            sum+=nums[i];
            maxsum = max(maxsum,sum);
            if(sum<0){
                sum=0;
            }
        }
        return maxsum;
    }
};
```

# 0065. 65. Valid Number

Hard • C++

## QUESTION

Given a string s, return whether s is a valid number. For example, all the following are valid numbers: "2", "0089", "-0.1", "+3.14", "4.", "-.9", "2e10", "-90E3", "3e+7", "+6e-1", "53.5e93", "-123.456e789", while the following are not valid numbers: "abc", "1a", "1e", "e3", "99e2.5", "--6", "-+3", "95a54e53".

Formally, a valid number is defined using one of the following definitions:

An integer number is defined with an optional sign '-' or '+' followed by digits.

A decimal number is defined with an optional sign '-' or '+' followed by one of the following definitions:

An exponent is defined with an exponent notation 'e' or 'E' followed by an integer number.

The digits are defined as one or more digits.

Input: s = "0"

Output: true

Input: s = "e"

Output: false

Input: s = "."

Output: false

## CONSTRAINTS

• 1 <= s.length <= 20

• s consists of only English letters (both uppercase and lowercase), digits (0-9), plus '+', minus '-', or dot '.'.

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    bool isNumber(string s) {
        bool num = false, dot = false, exp = false, expNum = true;
        for (int i = 0; i < s.size(); i++) {
            char c = s[i];
            if (isdigit(c)) {
                num = true;
                if (exp) expNum = true;
            }
            else if (c == '.') {
                if (dot || exp) return false;
                dot = true;
            }
            else if (c == 'e' || c == 'E') {
                if (exp || !num) return false;
                exp = true;
                expNum = false;
            }
            else if (c == '+' || c == '-') {
                if (i != 0 && s[i-1] != 'e' && s[i-1] != 'E')
                    return false;
            }
            else return false;
        }
        return num && expNum;
    }
};
```

# 0066. 66. Plus One

Easy • C++

## QUESTION

You are given a large integer represented as an integer array digits, where each digits[i] is the ith digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's.

Increment the large integer by one and return the resulting array of digits.

## EXAMPLES

### Example 1

```
Input: digits = [1,2,3]
Output: [1,2,4]
Explanation: The array represents the integer 123.
Incrementing by one gives 123 + 1 = 124.
Thus, the result should be [1,2,4].
```

### Example 2

```
Input: digits = [4,3,2,1]
Output: [4,3,2,2]
Explanation: The array represents the integer 4321.
Incrementing by one gives 4321 + 1 = 4322.
Thus, the result should be [4,3,2,2].
```

### Example 3

```
Input: digits = [9]
Output: [1,0]
Explanation: The array represents the integer 9.
Incrementing by one gives 9 + 1 = 10.
Thus, the result should be [1,0].
```

## CONSTRAINTS

- 1 <= digits.length <= 100
- 0 <= digits[i] <= 9
- digits does not contain any leading 0's.

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    vector<int> plusOne(vector<int>& digits) {
        for (int i = digits.size() - 1; i >= 0; i--) {
            if (digits[i] < 9) {
                digits[i]++;
                return digits;
            }
            digits[i] = 0;
        }
        digits.insert(digits.begin(), 1);
        return digits;
    }
};
```

# 0076. 76. Minimum Window Substring

Hard • C++

## QUESTION

Given two strings s and t of lengths m and n respectively, return the minimum window substring of s such that every character in t (including duplicates) is included in the window. If there is no such substring, return the empty string "".

The testcases will be generated such that the answer is unique.

Follow up: Could you find an algorithm that runs in O(m + n) time?

## EXAMPLES

### Example 1

```
Input: s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"
Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string t.
```

### Example 2

```
Input: s = "a", t = "a"
Output: "a"
Explanation: The entire string s is the minimum window.
```

### Example 3

```
Input: s = "a", t = "aa"
Output: ""
Explanation: Both 'a's from t must be included in the window.
Since the largest window of s only has one 'a', return empty string.
```

## CONSTRAINTS

• m == s.length

• n == t.length

• 1 <= m, n <= 105

• s and t consist of uppercase and lowercase English letters.

## COMPLEXITY

Time: O(n) average Space: O(n)

## SOLUTION

```cpp
class Solution {
public:
    string minWindow(string s, string t) {
        unordered_map<char,int>n,w;
        for(char c:t) n[c]++;
        int l = 0,start=0;
        int req = n.size();
        int have = 0;
        int len = INT_MAX ;
        for(int r = 0; r<s.size(); r++){
            w[s[r]]++;
            if(n.count(s[r]) && w[s[r]]==n[s[r]]){
                have++;
            }
            while(have == req){
                if(r-l+1 < len){
                    len = r-l+1;
                    start = l;
                }
                w[s[l]]--;
                if(n.count(s[l])&&w[s[l]]<n[s[l]]){
                    have--;
                }
                l++;
            }
        }
        if(len == INT_MAX) return "";
        return s.substr(start,len);

    }
};
```

# 0081. 81. Search in Rotated Sorted Array II

Medium • C++

## QUESTION

There is an integer array nums sorted in non-decreasing order (not necessarily with distinct values).

Before being passed to your function, nums is rotated at an unknown pivot index k (0 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed). For example, [0,1,2,4,4,4,5,6,6,7] might be rotated at pivot index 5 and become [4,5,6,6,7,0,1,2,4,4].

Given the array nums after the rotation and an integer target, return true if target is in nums, or false if it is not in nums.

You must decrease the overall operation steps as much as possible.

Follow up: This problem is similar to Search in Rotated Sorted Array, but nums may contain duplicates. Would this affect the runtime complexity? How and why?

## EXAMPLES

### Example 1

```
Input: nums = [2,5,6,0,0,1,2], target = 0
Output: true
```

### Example 2

```
Input: nums = [2,5,6,0,0,1,2], target = 3
Output: false
```

## CONSTRAINTS

• 1 <= nums.length <= 5000

• -104 <= nums[i] <= 104

• nums is guaranteed to be rotated at some pivot.

• -104 <= target <= 104

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    bool search(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target)
                return true;
            // Duplicates: cannot determine which half is sorted
            if (nums[left] == nums[mid] && nums[mid] == nums[right]) {
                left++;
                right--;
            }
            // Left half is sorted
            else if (nums[left] <= nums[mid]) {
                if (nums[left] <= target && target < nums[mid])
                    right = mid - 1;
                else
                    left = mid + 1;
            }
            // Right half is sorted
            else {
                if (nums[mid] < target && target <= nums[right])
                    left = mid + 1;
                else
                    right = mid - 1;
            }
        }
        return false;
    }
};
```

# 0094. 94. Binary Tree Inorder Traversal

Easy • C++

## QUESTION

Given the root of a binary tree, return the inorder traversal of its nodes' values.

Input: root = [1,null,2,3]

Output: [1,3,2]

Explanation:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,2,6,5,7,1,3,9,8]

Explanation:

Input: root = []

Output: []

Input: root = [1]

Output: [1]

## CONSTRAINTS

• The number of nodes in the tree is in the range [0, 100].

• -100 <= Node.val <= 100

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
 1  /**
 2   * Definition for a binary tree node.
 3   * struct TreeNode {
 4   *     int val;
 5   *     TreeNode *left;
 6   *     TreeNode *right;
 7   *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 8   *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 9   *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
10   * };
11   */
12  class Solution {
13  public:
14      vector<int> ans;
15      vector<int> inorderTraversal(TreeNode* root) {
16          if(root == NULL)
17              return ans;
18          inorderTraversal(root->left);
19          ans.push_back(root->val);
20          inorderTraversal(root->right);
21          return ans;
22      }
23  };
```

# 0098. 98. Validate Binary Search Tree

Medium • C++

## QUESTION

Given the root of a binary tree, determine if it is a valid binary search tree (BST).

A valid BST is defined as follows:

## EXAMPLES

### Example 1

```
Input: root = [2,1,3]
Output: true
```

### Example 2

```
Input: root = [5,1,4,null,null,3,6]
Output: false
Explanation: The root node's value is 5 but its right child's value is 4.
```

## CONSTRAINTS

• The number of nodes in the tree is in the range [1, 104].

• -231 <= Node.val <= 231 - 1

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
 1  /**
 2   * Definition for a binary tree node.
 3   * struct TreeNode {
 4   *     int val;
 5   *     TreeNode *left;
 6   *     TreeNode *right;
 7   *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 8   *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 9   *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
10   * };
11   */
12  class Solution {
13  public:
14      bool check(TreeNode* root,long long low,long long high){
15          if(root == NULL) return true;
16          if (root->val <= low || root->val >= high)
17              return false;
18          return check(root->left,low,root->val)&&check(root->right,root->val,high);
19      }
20      bool isValidBST(TreeNode* root) {
21          return check(root,LLONG_MIN,LLONG_MAX);
22      }
23  };
```

# 0100. 100. Same Tree

Easy • C++

## QUESTION

Given the roots of two binary trees p and q, write a function to check if they are the same or not.

Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

## EXAMPLES

### Example 1

```
Input: p = [1,2,3], q = [1,2,3]
Output: true
```

### Example 2

```
Input: p = [1,2], q = [1,null,2]
Output: false
```

### Example 3

```
Input: p = [1,2,1], q = [1,1,2]
Output: false
```

## CONSTRAINTS

• The number of nodes in both trees is in the range [0, 100].

• -104 <= Node.val <= 104

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
 1  /**
 2   * Definition for a binary tree node.
 3   * struct TreeNode {
 4   *     int val;
 5   *     TreeNode *left;
 6   *     TreeNode *right;
 7   *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 8   *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 9   *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
10   * };
11   */
12  class Solution {
13  public:
14      bool isSameTree(TreeNode* p, TreeNode* q) {
15          if(p==NULL && q==NULL)return true;
16          if(p==NULL || q==NULL)return false;
17          if(p->val!=q->val) return false;
18          return isSameTree(p->left,q->left)&&isSameTree(p->right,q->right);
19      }
20  };
```

# 0101. 101. Symmetric Tree

Easy • C++

## QUESTION

Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

## EXAMPLES

### Example 1

```
Input: root = [1,2,2,3,4,4,3]
Output: true
```

### Example 2

```
Input: root = [1,2,2,null,3,null,3]
Output: false
```

## CONSTRAINTS

• The number of nodes in the tree is in the range [1, 1000].

• -100 <= Node.val <= 100

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
 1  /**
 2   * Definition for a binary tree node.
 3   * struct TreeNode {
 4   *     int val;
 5   *     TreeNode *left;
 6   *     TreeNode *right;
 7   *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 8   *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 9   *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
10   * };
11   */
12  class Solution {
13  public:
14      bool mirror(TreeNode* left, TreeNode* right) {
15          if (left == NULL && right == NULL)
16              return true;
17          if (left == NULL || right == NULL)
18              return false;
19          if (left->val != right->val)
20              return false;
21          return mirror(left->left, right->right) &&
22                 mirror(left->right, right->left);
23      }
24      bool isSymmetric(TreeNode* root) {
25          if (root == NULL)
26              return true;
27          return mirror(root->left, root->right);
28      }
29  };
```

# 0121. 121. Best Time to Buy and Sell Stock

Easy • C++

## QUESTION

You are given an array prices where prices[i] is the price of a given stock on the ith day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

## EXAMPLES

### Example 1

```
Input: prices = [7,1,5,3,6,4]
Output: 5
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sel
```

### Example 2

```
Input: prices = [7,6,4,3,1]
Output: 0
Explanation: In this case, no transactions are done and the max profit = 0.
```

## CONSTRAINTS

• 1 <= prices.length <= 105

• 0 <= prices[i] <= 104

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
 1  class Solution {
 2  public:
 3      int maxProfit(vector<int>& prices) {
 4          int minprice = prices[0];
 5          int profit = 0;
 6          for(int i =1;i<prices.size();i++){
 7              minprice= min(minprice,prices[i]);
 8              profit = max(profit,prices[i]-minprice);
 9          }
10          return profit;
11      }
12  };
```

# 0122. 122. Best Time to Buy and Sell Stock II

Medium • C++

## QUESTION

You are given an integer array prices where prices[i] is the price of a given stock on the ith day.

On each day, you may decide to buy and/or sell the stock. You can only hold at most one share of the stock at any time. However, you can sell and buy the stock multiple times on the same day, ensuring you never hold more than one share of the stock.

Find and return the maximum profit you can achieve.

## EXAMPLES

### Example 1

```
Input: prices = [7,1,5,3,6,4]
Output: 7
Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = 5-1 = 4.
Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = 6-3 = 3.
Total profit is 4 + 3 = 7.
```

### Example 2

```
Input: prices = [1,2,3,4,5]
Output: 4
Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.
Total profit is 4.
```

### Example 3

```
Input: prices = [7,6,4,3,1]
Output: 0
Explanation: There is no way to make a positive profit, so we never buy the stock to achieve the
```

## CONSTRAINTS

• 1 <= prices.length <= 3 * 104

• 0 <= prices[i] <= 104

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int profit=0;
        for(int i = 1;i<prices.size();i++){
            if(prices[i]>prices[i-1]){
                profit = profit + prices[i] - prices[i-1];
            }
        }
        return profit;
    }
};
```

# 0128. 128. Longest Consecutive Sequence

Medium • C++

## QUESTION

Given an unsorted array of integers nums, return the length of the longest consecutive elements sequence.

You must write an algorithm that runs in O(n) time.

## EXAMPLES

### Example 1

```
Input: nums = [100,4,200,1,3,2]
Output: 4
Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4
```

### Example 2

```
Input: nums = [0,3,7,2,5,8,4,6,0,1]
Output: 9
```

### Example 3

```
Input: nums = [1,0,1,2]
Output: 3
```

## CONSTRAINTS

• 0 <= nums.length <= 105

• -109 <= nums[i] <= 109

## COMPLEXITY

Time: O(n) average Space: O(n)

## SOLUTION

```cpp
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> st(nums.begin(),nums.end());
        int ans =0;
        for(int x:st){
            if(st.find(x-1)==st.end()){
                int current =x;
                int count =1;
                while(st.find(current+1)!=st.end()){
                    current++;
                    count++;
                }
                ans=max(ans,count);
            }
        }
        return ans;
    }
};
```

# 0135. 135. Candy

Hard • C++

## QUESTION

There are n children standing in a line.

Each child is assigned a rating value given in the integer array ratings.

You are giving candies to these children subjected to the following requirements:

Return the minimum number of candies you need to have to distribute the candies to the children.

## EXAMPLES

### Example 1

```
Input: ratings = [1,0,2]
Output: 5
Explanation: You can allocate to the first, second and third child with 2, 1, 2 candies respectiv
```

### Example 2

```
Input: ratings = [1,2,2]
Output: 4
Explanation: You can allocate to the first, second and third child with 1, 2, 1 candies respectiv
The third child gets 1 candy because it satisfies the above two conditions.
```

## CONSTRAINTS

• 1 <= n == ratings.length <= 5 * 104

• 0 <= ratings[i] <= 5 * 104

## COMPLEXITY

Time: O(n²) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    int candy(vector<int>& ratings) {
        int n = ratings.size();
        vector<int> candy(n, 1);
        for(int i = 1; i < n; i++) {
            if(ratings[i] > ratings[i-1])
                candy[i] = candy[i-1] + 1;
        }
        for(int i = n-2; i >= 0; i--) {
            if(ratings[i] > ratings[i+1])
                candy[i] = max(candy[i], candy[i+1] + 1);
        }
        int count = 0;
        for(int x : candy)
            count += x;
        return count;
    }
};
```

# 0155. 155. Min Stack

Medium • C++

## QUESTION

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

You must implement a solution with O(1) time complexity for each function.

## EXAMPLES

### Example 1

```
Input
["MinStack","push","push","push","getMin","pop","top","getMin"]
[[],[-2],[0],[-3],[],[],[],[]]

Output
[null,null,null,null,-3,null,0,-2]

Explanation
MinStack minStack = new MinStack();
minStack.push(-2);
minStack.push(0);
minStack.push(-3);
minStack.getMin(); // return -3
minStack.pop();
minStack.top(); // return 0
minStack.getMin(); // return -2
```

## CONSTRAINTS

- -231 <= val <= 231 - 1
- Methods pop, top and getMin operations will always be called on non-empty stacks.
- At most 3 * 104 calls will be made to push, pop, top, and getMin.

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
class MinStack {
public:
    stack<int> st;
    stack<int> mn;
    MinStack() {
    }
    void push(int val) {
        st.push(val);
        if (mn.empty() || val <= mn.top())
            mn.push(val);
    }
    void pop() {
        if (st.top() == mn.top())
            mn.pop();
        st.pop();
    }
    int top() {
        return st.top();
    }
    int getMin() {
        return mn.top();
    }
};
```

# 0189. 189. Rotate Array

Medium • C++

## QUESTION

Given an integer array nums, rotate the array to the right by k steps, where k is non-negative.

Follow up:

## EXAMPLES

### Example 1

```
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
```

### Example 2

```
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
```

## CONSTRAINTS

• 1 <= nums.length <= 105

• -231 <= nums[i] <= 231 - 1

• 0 <= k <= 105

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    void rotate(vector<int>& nums, int k) {
        k = k % nums.size();
        reverse(nums.begin(), nums.end());
        reverse(nums.begin(), nums.begin() + k);
        reverse(nums.begin() + k, nums.end());
    }
};
```

# 0209. 209. Minimum Size Subarray Sum

Medium • C++

## QUESTION

Given an array of positive integers nums and a positive integer target, return the minimal length of a subarray whose sum is greater than or equal to target. If there is no such subarray, return 0 instead.

## EXAMPLES

### Example 1

```
Input: target = 7, nums = [2,3,1,2,4,3]
Output: 2
Explanation: The subarray [4,3] has the minimal length under the problem constraint.
```

### Example 2

```
Input: target = 4, nums = [1,4,4]
Output: 1
```

### Example 3

```
Input: target = 11, nums = [1,1,1,1,1,1,1,1]
Output: 0
```

## CONSTRAINTS

- 1 <= target <= 109
- 1 <= nums.length <= 105
- 1 <= nums[i] <= 104

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int l=0;
        int sum=0;
        int len = INT_MAX;
        for(int r=0;r<nums.size();++r){
            sum += nums[r];
            while(sum>=target){
                len = min(r-l+1,len);
                sum-=nums[l];
                ++l;
            }
        }
        if(len == INT_MAX) return 0;
        return len;
    }
};
```

# 0215. 215. Kth Largest Element in an Array

Medium • C++

## QUESTION

Given an integer array nums and an integer k, return the kth largest element in the array.

Note that it is the kth largest element in the sorted order, not the kth distinct element.

Can you solve it without sorting?

## EXAMPLES

### Example 1

```
Input: nums = [3,2,1,5,6,4], k = 2
Output: 5
```

### Example 2

```
Input: nums = [3,2,3,1,2,4,5,5,6], k = 4
Output: 4
```

## CONSTRAINTS

• 1 <= k <= nums.length <= 105

• -104 <= nums[i] <= 104

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
 1  class Solution {
 2  public:
 3      int findKthLargest(vector<int>& nums, int k) {
 4          priority_queue<int, vector<int> ,greater<int>> pq;
 5          for(int x:nums){
 6              pq.push(x);
 7              if(pq.size()>k){
 8                  pq.pop();
 9              }
10          }
11          return pq.top();
12      }
13  };
```

# 0239. 239. Sliding Window Maximum

Hard • C++

## QUESTION

You are given an array of integers nums, there is a sliding window of size k which is moving from the very left of the array to the very right. You can only see the k numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window.

## EXAMPLES

### Example 1

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]
Explanation:
Window position Max
--------------- -----
[1 3 -1] -3 5 3 6 7 3
 1 [3 -1 -3] 5 3 6 7 3
 1 3 [-1 -3 5] 3 6 7 5
 1 3 -1 [-3 5 3] 6 7 5
 1 3 -1 -3 [5 3 6] 7 6
 1 3 -1 -3 5 [3 6 7] 7
```

### Example 2

```
Input: nums = [1], k = 1
Output: [1]
```

## CONSTRAINTS

• 1 <= nums.length <= 105

• -104 <= nums[i] <= 104

• 1 <= k <= nums.length

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        deque<int> dq;
        vector<int> ans;
        for (int r = 0; r < nums.size(); r++) {
            // Remove elements outside the window
            while (!dq.empty() && dq.front() <= r - k) {
                dq.pop_front();
            }
            // Remove smaller elements
            while (!dq.empty() && nums[dq.back()] <= nums[r]) {
                dq.pop_back();
            }
            // Add current index
            dq.push_back(r);
            // Window size is k
            if (r >= k - 1) {
                ans.push_back(nums[dq.front()]);
            }
        }
        return ans;
    }
};
```

# 0274. 274. H-Index

Medium • C++

## QUESTION

Given an array of integers citations where citations[i] is the number of citations a researcher received for their ith paper, return the researcher's h-index.

According to the definition of h-index on Wikipedia: The h-index is defined as the maximum value of h such that the given researcher has published at least h papers that have each been cited at least h times.

## EXAMPLES

### Example 1

```
Input: citations = [3,0,6,1,5]
Output: 3
Explanation: [3,0,6,1,5] means the researcher has 5 papers in total and each of them had received
Since the researcher has 3 papers with at least 3 citations each and the remaining two with no mo
```

### Example 2

```
Input: citations = [1,3,1]
Output: 1
```

## CONSTRAINTS

- n == citations.length
- 1 <= n <= 5000
- 0 <= citations[i] <= 1000

## COMPLEXITY

Time: O(n log n) Space: O(n)

## SOLUTION

```cpp

class Solution {
public:
    int hIndex(vector<int>& citations) {
        sort(citations.begin(), citations.end());
        int n = citations.size();
        for (int i = 0; i < n; i++) {
            int h = n - i;
            if (citations[i] >= h)
                return h;
        }
        return 0;
    }
};
```

# 0283. 283. Move Zeroes

Easy • C++

## QUESTION

Given an integer array nums, move all 0's to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

## EXAMPLES

### Example 1

```
Input: nums = [0,1,0,3,12]
Output: [1,3,12,0,0]
```

### Example 2

```
Input: nums = [0]
Output: [0]
```

## CONSTRAINTS

• 1 <= nums.length <= 104

• -231 <= nums[i] <= 231 - 1

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
 1  class Solution {
 2  public:
 3      void moveZeroes(vector<int>& nums) {
 4          int j = 0;
 5          for (int i = 0; i < nums.size(); i++) {
 6              if (nums[i] != 0) {
 7                  swap(nums[i], nums[j]);
 8                  j++;
 9              }
10          }
11      }
12  };
```

# 0342. 342. Power of Four

Easy • C++

## QUESTION

Given an integer n, return true if it is a power of four. Otherwise, return false.

An integer n is a power of four, if there exists an integer x such that n == 4x.

## EXAMPLES

### Example 1

```
Input: n = 16
Output: true
```

### Example 2

```
Input: n = 5
Output: false
```

### Example 3

```
Input: n = 1
Output: true
```

## CONSTRAINTS

• -231 <= n <= 231 - 1

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
 1  class Solution {
 2  public:
 3      bool isPowerOfFour(int n) {
 4          if(n<=0) return false;
 5          while(n%4==0){
 6              n/=4;
 7          }
 8          return n==1;
 9      }
10  };
```

# 0455. 455. Assign Cookies

Easy • C++

## QUESTION

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Note: This question is the same as 2410: Maximum Matching of Players With Trainers.

## EXAMPLES

### Example 1

```
Input: g = [1,2,3], s = [1,1]
Output: 1
Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.
And even though you have 2 cookies, since their size is both 1, you could only make the child who
You need to output 1.
```

### Example 2

```
Input: g = [1,2], s = [1,2,3]
Output: 2
Explanation: You have 2 children and 3 cookies. The greed factors of 2 children are 1, 2.
You have 3 cookies and their sizes are big enough to gratify all of the children,
You need to output 2.
```

## CONSTRAINTS

- 1 <= g.length <= 3 * 104
- 0 <= s.length <= 3 * 104
- 1 <= g[i], s[j] <= 231 - 1

## COMPLEXITY

Time: O(n log n) Space: O(n)

## SOLUTION

```cpp
class Solution {
public:
    int findContentChildren(vector<int>& g, vector<int>& s) {
        sort(g.begin(), g.end());
        sort(s.begin(), s.end());
        int i = 0;
        int j = 0;
        int count = 0;
        while (i < g.size() && j < s.size()) {
            if (s[j] >= g[i]) {
                count++;
                i++;
                j++;
            }
            else {
                j++;
            }
        }
        return count;
    }
};
```

# 0628. 628. Maximum Product of Three Numbers

Easy • C++

## QUESTION

You are given an integer array nums.

Find three numbers whose product is maximum and return the maximum product.

Input: nums = [1,2,3]

Output: 6

Explanation:

The only three numbers are 1, 2, and 3, so the maximum product is 1 * 2 * 3 = 6.

Input: nums = [1,2,3,4]

Output: 24

Explanation:

The largest product comes from the three greatest numbers: 2 * 3 * 4 = 24.

Input: nums = [-1,-2,-3]

Output: -6

Explanation:

The only three numbers are -1, -2, and -3, so the maximum product is (-1) * (-2) * (-3) = -6.

## CONSTRAINTS

• 3 <= nums.length <= 104

• -1000 <= nums[i] <= 1000

## COMPLEXITY

Time: O(n log n) Space: O(n)

## SOLUTION

```
 1  class Solution {
 2  public:
 3      int maximumProduct(vector<int>& nums) {
 4          sort(nums.begin(), nums.end());
 5          int n = nums.size();
 6          int a = nums[n-1] * nums[n-2] * nums[n-3];
 7          int b = nums[0] * nums[1] * nums[n-1];
 8          return max(a, b);
 9      }
10  };
```

# 0701. 784. Insert into a Binary Search Tree

Medium • C++

## QUESTION

You are given the root node of a binary search tree (BST) and a value to insert into the tree. Return the root node of the BST after the insertion. It is guaranteed that the new value does not exist in the original BST.

Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return any of them.

## EXAMPLES

### Example 1

```
Input: root = [4,2,7,1,3], val = 5
Output: [4,2,7,1,3,5]
Explanation: Another accepted tree is:
```

### Example 2

```
Input: root = [40,20,60,10,30,50,70], val = 25
Output: [40,20,60,10,30,50,70,null,null,25]
```

### Example 3

```
Input: root = [4,2,7,1,3,null,null,null,null,null,null], val = 5
Output: [4,2,7,1,3,5]
```

## CONSTRAINTS

- The number of nodes in the tree will be in the range [0, 104].
- -108 <= Node.val <= 108
- All the values Node.val are unique.
- -108 <= val <= 108
- It's guaranteed that val does not exist in the original BST.

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
 1  /**
 2   * Definition for a binary tree node.
 3   * struct TreeNode {
 4   *     int val;
 5   *     TreeNode *left;
 6   *     TreeNode *right;
 7   *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 8   *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 9   *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
10   * };
11   */
12  class Solution {
13  public:
14      TreeNode* insertIntoBST(TreeNode* root, int val) {
15          // Empty position found
16          if(root == NULL) {
17              return new TreeNode(val);
18          }
19          // Go right
20          if(val > root->val) {
21              root->right = insertIntoBST(root->right, val);
22          }
23          // Go left
24          else {
25              root->left = insertIntoBST(root->left, val);
26          }
27          return root;
28      }
29  };
```

# 0724. 724. Find Pivot Index

Easy • C++

## QUESTION

Given an array of integers nums, calculate the pivot index of this array.

The pivot index is the index where the sum of all the numbers strictly to the left of the index is equal to the sum of all the numbers strictly to the index's right.

If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array.

Return the leftmost pivot index. If no such index exists, return -1.

Note: This question is the same as 1991: https://leetcode.com/problems/find-the-middle-index-in-array/

## EXAMPLES

### Example 1

```
Input: nums = [1,7,3,6,5,6]
Output: 3
Explanation:
The pivot index is 3.
Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11
Right sum = nums[4] + nums[5] = 5 + 6 = 11
```

### Example 2

```
Input: nums = [1,2,3]
Output: -1
Explanation:
There is no index that satisfies the conditions in the problem statement.
```

### Example 3

```
Input: nums = [2,1,-1]
Output: 0
Explanation:
The pivot index is 0.
Left sum = 0 (no elements to the left of index 0)
Right sum = nums[1] + nums[2] = 1 + -1 = 0
```

## CONSTRAINTS

• 1 <= nums.length <= 104

• -1000 <= nums[i] <= 1000

## COMPLEXITY

Time: $O(n^2)$ Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    int pivotIndex(vector<int>& nums) {
        int total = 0;
        for (int x : nums)
            total += x;
        int leftSum = 0;
        for (int i = 0; i < nums.size(); i++) {
            int rightSum = total - leftSum - nums[i];
            if (leftSum == rightSum)
                return i;
            leftSum += nums[i];
        }
        return -1;
    }
};
```

# 0796. 812. Rotate String

Easy • C++

## QUESTION

Given two strings s and goal, return true if and only if s can become goal after some number of shifts on s.

A shift on s consists of moving the leftmost character of s to the rightmost position.

## EXAMPLES

### Example 1

```
Input: s = "abcde", goal = "cdeab"
Output: true
```

### Example 2

```
Input: s = "abcde", goal = "abced"
Output: false
```

## CONSTRAINTS

• 1 <= s.length, goal.length <= 100

• s and goal consist of lowercase English letters.

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    bool rotateString(string s, string goal) {
        if (s.length() != goal.length())
            return false;
        string x = s + s;
        return x.find(goal) != string::npos;
    }
};
```

# 0896. 932. Monotonic Array

Easy • C++

## QUESTION

An array is monotonic if it is either monotone increasing or monotone decreasing.

An array nums is monotone increasing if for all i <= j, nums[i] <= nums[j]. An array nums is monotone decreasing if for all i <= j, nums[i] >= nums[j].

Given an integer array nums, return true if the given array is monotonic, or false otherwise.

## EXAMPLES

### Example 1

```
Input: nums = [1,2,2,3]
Output: true
```

### Example 2

```
Input: nums = [6,5,4,4]
Output: true
```

### Example 3

```
Input: nums = [1,3,2]
Output: false
```

## CONSTRAINTS

• 1 <= nums.length <= 105

• -105 <= nums[i] <= 105

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```
 1  class Solution {
 2  public:
 3      bool isMonotonic(vector<int>& nums) {
 4          bool increasing = true;
 5          bool decreasing = true;
 6          for(int i = 0; i < nums.size() - 1; i++) {
 7              if(nums[i] > nums[i+1])
 8                  increasing = false;
 9              if(nums[i] < nums[i+1])
10                  decreasing = false;
11          }
12          return increasing || decreasing;
13      }
14  };
```

# 1051. 1137. Height Checker

Easy • C++

## QUESTION

A school is trying to take an annual photo of all the students. The students are asked to stand in a single file line in non-decreasing order by height. Let this ordering be represented by the integer array expected where expected[i] is the expected height of the ith student in line.

You are given an integer array heights representing the current order that the students are standing in. Each heights[i] is the height of the ith student in line (0-indexed).

Return the number of indices where heights[i] != expected[i].

## EXAMPLES

### Example 1

```
Input: heights = [1,1,4,2,1,3]
Output: 3
Explanation:
heights: [1,1,4,2,1,3]
expected: [1,1,1,2,3,4]
Indices 2, 4, and 5 do not match.
```

### Example 2

```
Input: heights = [5,1,2,3,4]
Output: 5
Explanation:
heights: [5,1,2,3,4]
expected: [1,2,3,4,5]
All indices do not match.
```

### Example 3

```
Input: heights = [1,2,3,4,5]
Output: 0
Explanation:
heights: [1,2,3,4,5]
expected: [1,2,3,4,5]
All indices match.
```

## CONSTRAINTS

• 1 <= heights.length <= 100

• 1 <= heights[i] <= 100

## COMPLEXITY

Time: O(n log n) Space: O(n)

## SOLUTION

```cpp
class Solution {
public:
    int heightChecker(vector<int>& heights) {
        int count =0;
        vector<int> copy=heights;
        sort(copy.begin(),copy.end());
        for(int i=0;i<heights.size();i++){
            if(copy[i]!=heights[i]){
                count++;
            }
        }
        return count;
    }
};
```

# 1438. 1549. Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit

Medium • C++

## QUESTION

Given an array of integers nums and an integer limit, return the size of the longest non-empty subarray such that the absolute difference between any two elements of this subarray is less than or equal to limit.

## EXAMPLES

### Example 1

```
Input: nums = [8,2,4,7], limit = 4
Output: 2
Explanation: All subarrays are:
[8] with maximum absolute diff |8-8| = 0 <= 4.
[8,2] with maximum absolute diff |8-2| = 6 > 4.
[8,2,4] with maximum absolute diff |8-2| = 6 > 4.
[8,2,4,7] with maximum absolute diff |8-2| = 6 > 4.
[2] with maximum absolute diff |2-2| = 0 <= 4.
[2,4] with maximum absolute diff |2-4| = 2 <= 4.
[2,4,7] with maximum absolute diff |2-7| = 5 > 4.
[4] with maximum absolute diff |4-4| = 0 <= 4.
[4,7] with maximum absolute diff |4-7| = 3 <= 4.
[7] with maximum absolute diff |7-7| = 0 <= 4.
Therefore, the size of the longest subarray is 2.
```

### Example 2

```
Input: nums = [10,1,2,4,7,2], limit = 5
Output: 4
Explanation: The subarray [2,4,7,2] is the longest since the maximum absolute diff is |2-7| = 5 <
```

### Example 3

```
Input: nums = [4,2,2,2,4,4,2,2], limit = 0
Output: 3
```

## CONSTRAINTS

- 1 <= nums.length <= 105
- 1 <= nums[i] <= 109
- 0 <= limit <= 109

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    int longestSubarray(vector<int>& nums, int limit) {
        deque<int> maxi;
        deque<int> mini;
        int l = 0;
        int ans = 0;
        for (int r = 0; r < nums.size(); r++) {
            // Maintain decreasing deque for maximum
            while (!maxi.empty() && nums[maxi.back()] < nums[r]) {
                maxi.pop_back();
            }
            maxi.push_back(r);
            // Maintain increasing deque for minimum
            while (!mini.empty() && nums[mini.back()] > nums[r]) {
                mini.pop_back();
            }
            mini.push_back(r);
            // Window invalid
            while (nums[maxi.front()] - nums[mini.front()] > limit) {
                if (maxi.front() == l)
                    maxi.pop_front();
                if (mini.front() == l)
                    mini.pop_front();
                l++;
            }
            ans = max(ans, r - l + 1);
        }
        return ans;
    }
};
```

# 1980. 2107. Find Unique Binary String

Medium • C++

## QUESTION

Given an array of strings nums containing n unique binary strings each of length n, return a binary string of length n that does not appear in nums. If there are multiple answers, you may return any of them.

## EXAMPLES

### Example 1

```
Input: nums = ["01","10"]
Output: "11"
Explanation: "11" does not appear in nums. "00" would also be correct.
```

### Example 2

```
Input: nums = ["00","01"]
Output: "11"
Explanation: "11" does not appear in nums. "10" would also be correct.
```

### Example 3

```
Input: nums = ["111","011","001"]
Output: "101"
Explanation: "101" does not appear in nums. "000", "010", "100", and "110" would also be correct.
```

## CONSTRAINTS

- n == nums.length
- 1 <= n <= 16
- nums[i].length == n
- nums[i] is either '0' or '1'.
- All the strings of nums are unique.

## COMPLEXITY

Time: O(n) Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    string findDifferentBinaryString(vector<string>& nums) {
        string ans = "";
        for (int i = 0; i < nums.size(); i++) {
            if (nums[i][i] == '0')
                ans += '1';
            else
                ans += '0';
        }
        return ans;
    }
};
```

# 2426. 2513. Number of Pairs Satisfying Inequality

Hard • C++

## QUESTION

You are given two 0-indexed integer arrays nums1 and nums2, each of size n, and an integer diff. Find the number of pairs (i, j) such that:

Return the number of pairs that satisfy the conditions.

## EXAMPLES

### Example 1

```
Input: nums1 = [3,2,5], nums2 = [2,2,1], diff = 1
Output: 3
Explanation:
There are 3 pairs that satisfy the conditions:
1. i = 0, j = 1: 3 - 2 <= 2 - 2 + 1. Since i < j and 1 <= 1, this pair satisfies the conditions.
2. i = 0, j = 2: 3 - 5 <= 2 - 1 + 1. Since i < j and -2 <= 2, this pair satisfies the conditions.
3. i = 1, j = 2: 2 - 5 <= 2 - 1 + 1. Since i < j and -3 <= 2, this pair satisfies the conditions.
Therefore, we return 3.
```

### Example 2

```
Input: nums1 = [3,-1], nums2 = [-2,2], diff = -1
Output: 0
Explanation:
Since there does not exist any pair that satisfies the conditions, we return 0.
```

## CONSTRAINTS

- n == nums1.length == nums2.length
- 2 <= n <= 105
- -104 <= nums1[i], nums2[i] <= 104
- -104 <= diff <= 104

## COMPLEXITY

Time: $O(n^2)$ Space: O(1)

## SOLUTION

```cpp
class Solution {
public:
    long long mergeSort(vector<long long>& a, int l, int r, int diff) {
        if(l >= r) return 0;
        int mid = l + (r - l) / 2;
        long long count = 0;
        count += mergeSort(a, l, mid, diff);
        count += mergeSort(a, mid + 1, r, diff);
        // Count pairs: i in left, j in right
        int j = mid + 1;
        for(int i = l; i <= mid; i++) {
            while(j <= r && a[i] > a[j] + diff)
                j++;
            count += r - j + 1;
        }
        // Normal merge
        vector<long long> temp;
        int i = l;
        j = mid + 1;
        while(i <= mid && j <= r) {
            if(a[i] <= a[j])
                temp.push_back(a[i++]);
            else
                temp.push_back(a[j++]);
        }
        while(i <= mid)
            temp.push_back(a[i++]);
        while(j <= r)
            temp.push_back(a[j++]);
        for(int k = l; k <= r; k++)
            a[k] = temp[k - l];
        return count;
    }
    long long numberOfPairs(vector<int>& nums1,vector<int>& nums2,int diff) {
        int n = nums1.size();
        vector<long long> a(n);
        for(int i = 0; i < n; i++)
            a[i] = (long long)nums1[i] - nums2[i];
        return mergeSort(a, 0, n - 1, diff);
    }
};
```

# 3718. 4080. Smallest Missing Multiple of K

Easy • C++

## QUESTION

Given an integer array nums and an integer k, return the smallest positive multiple of k that is missing from nums.

A multiple of k is any positive integer divisible by k.

Input: nums = [8,2,3,4,6], k = 2

Output: 10

Explanation:

The multiples of k = 2 are 2, 4, 6, 8, 10, 12... and the smallest multiple missing from nums is 10.

Input: nums = [1,4,7,10,15], k = 5

Output: 5

Explanation:

The multiples of k = 5 are 5, 10, 15, 20... and the smallest multiple missing from nums is 5.

## CONSTRAINTS

- 1 <= nums.length <= 100
- 1 <= nums[i] <= 100
- 1 <= k <= 100

## COMPLEXITY

Time: O(n) average Space: O(n)

## SOLUTION

```cpp
class Solution {
public:
    int missingMultiple(vector<int>& nums, int k) {
        unordered_set<int> st;
        for(int x : nums){
            st.insert(x);
        }
        for(int i=1;i<=nums.size()+1;i++){
            int x = k*i;
            if(st.find(x)==st.end()){
                return x;
            }
        }
        return -1;
    }
};
```

# 3867. 4242. Sum of GCD of Formed Pairs

Medium • C++

## QUESTION

You are given an integer array nums of length n.

Construct an array prefixGcd where for each index i:

After constructing prefixGcd:

Return an integer denoting the sum of the GCD values of all formed pairs.

Input: nums = [2,6,4]

Output: 2

Explanation:

Construct prefixGcd:

prefixGcd = [2, 6, 2]. After sorting, it forms [2, 2, 6].

Pair the smallest and largest elements: gcd(2, 6) = 2. The remaining middle element 2 is ignored. Thus, the sum is 2.

Input: nums = [3,6,2,8]

Output: 5

Explanation:

Construct prefixGcd:

prefixGcd = [3, 6, 2, 8]. After sorting, it forms [2, 3, 6, 8].

Form pairs: gcd(2, 8) = 2 and gcd(3, 6) = 3. Thus, the sum is 2 + 3 = 5.

## CONSTRAINTS

• 1 <= n == nums.length <= 105

• 1 <= nums[i] <= 109

## COMPLEXITY

Time: O(n log n) Space: O(n)

## SOLUTION

```cpp
class Solution {
public:
    long long gcdSum(vector<int>& nums) {
        vector<int> g;
        int m = 0;
        int a = 0;
        for(int i = 0; i < nums.size(); i++){
            m = max(m, nums[i]);
            a = gcd(nums[i], m);
            g.push_back(a);
        }
        long long sum = 0;
        int b = 0;
        sort(g.begin(), g.end());
        for(int i = 0; i < g.size()/2; i++){
            b = gcd(g[i], g[g.size()-1-i]);
            sum += b;
        }
        return sum;
    }
};
```